

UNIVERSIDADE FEDERAL DE MATO GROSSO DO SUL
FACOM - FACULDADE DE COMPUTAÇÃO | ENGENHARIA DE SOFTWARE

ESPECIFICAÇÃO DE REQUISITOS DE SOFTWARE

**MANGO TRADING: UM JOGO QUE SIMULA O MERCADO
FINANCEIRO**

MANTENEDORES:

Abelardo Palácios Ribeiro

Miguel Ribeiro Bernal

Giovana Gomes Pavese

Campo Grande – Mato Grosso do Sul

2026

PREFÁCIO

Este Documento de Requisitos é destinado aos interessados diretos e indiretos no desenvolvimento do sistema Mango Trading, uma aplicação web de simulação de trading com ações de empresas fictícias. Entre os principais públicos-alvo estão a equipe de desenvolvimento responsável pelo back-end e pelo front-end da aplicação, os orientadores e avaliadores acadêmicos e os demais gestores envolvidos no projeto.

O objetivo deste documento é registrar as funcionalidades, os requisitos, as regras de negócio e as demais características já definidas para o sistema, servindo como base para as fases subsequentes do projeto, incluindo análise, especificação, arquitetura, implementação e testes.

HISTÓRICO DE REVISÕES

Versão	Data	Autor	Descrição
0.1	08/12/2006	Equipe de Requisitos	Elaboração da versão inicial baseada no padrão IEEE 830.
1.0	14/09/2026	Equipe Mango Trading	Especificação completa do projeto Mango Trading, elaborada a partir do documento de planejamento do Simulador de Trading com Mercado de Ações Fictícias.
1.1	14/09/2026	Equipe Mango Trading	Inclusão do diagrama de casos de uso e ajustes de formatação, alinhando o documento à atividade de Entrega do Protótipo (Esqueleto).

1. INTRODUÇÃO

1.1 Propósito

O propósito deste documento é apresentar a Especificação de Requisitos de Software (ERS) para o sistema Mango Trading, descrevendo formalmente suas funcionalidades, tecnologias, arquitetura, requisitos e demais características definidas no planejamento do projeto. Este documento é destinado à equipe de desenvolvimento, aos orientadores acadêmicos e aos avaliadores responsáveis pelo projeto de Engenharia de Software.

1.2 Escopo

O Mango Trading é uma aplicação web de simulação de trading, na qual os usuários podem comprar e vender ações de empresas fictícias utilizando dinheiro virtual. A aplicação simula, de forma simplificada, o funcionamento de um mercado financeiro: os jogadores iniciam com um saldo virtual e podem investir em diferentes empresas fictícias, acompanhar a valorização ou a desvalorização das ações, gerenciar sua carteira de investimentos e competir com outros usuários por meio de um sistema de ranking.

O projeto utiliza uma arquitetura desacoplada, composta por um back-end responsável pela API e pelas regras de negócio e um front-end separado responsável pela interface e pela interação com os usuários.

O que o sistema faz:

- Permite o cadastro e a autenticação de usuários.
- Implementa diferentes níveis de acesso (Usuário Comum e Administrador).
- Permite a administração de empresas e ações fictícias.
- Permite a compra e a venda de ações utilizando saldo virtual.
- Permite o acompanhamento da carteira de investimentos.
- Registra todas as transações realizadas.
- Calcula o patrimônio total dos usuários.
- Apresenta um ranking competitivo entre os jogadores.

O que o sistema NÃO faz:

- Não utiliza dinheiro real em nenhuma operação de compra ou venda.
- Não negocia empresas ou ações reais; todas as empresas, ações, preços e eventos de mercado são fictícios.

Objetivos e Benefícios: além de simular o funcionamento de um mercado financeiro, o projeto tem como objetivo demonstrar conhecimentos relacionados a desenvolvimento web, APIs REST, segurança, persistência de dados, arquitetura de software, acessibilidade e trabalho em equipe, atendendo aos requisitos do trabalho acadêmico de Engenharia de Software.

1.3 Definições, Acrônimos e Abreviações

Termo	Definição
MVP	Minimum Viable Product (Produto Mínimo Viável): primeira versão funcional do sistema, contendo o conjunto essencial de funcionalidades definidas no planejamento do projeto.
JWT	JSON Web Token: padrão utilizado pelo Spring Security para autenticação da API.
API REST	Interface de comunicação entre o front-end e o back-end, baseada em requisições HTTP e dados no formato JSON.
CRUD	Conjunto de operações de criação, leitura, atualização e remoção (Create, Read, Update, Delete), utilizado, por exemplo, na administração de empresas e ações.
Carteira	Conjunto de ações adquiridas por um usuário, contendo informações de quantidade, preço médio de compra, preço atual e valor investido.

Termo	Definição
Patrimônio Total	Soma do saldo disponível do usuário com o valor atual das ações presentes em sua carteira.
Ranking	Classificação dos usuários de acordo com o patrimônio total, permitindo a comparação de desempenho entre os jogadores.

1. Visão Geral do Documento

Este documento está estruturado em cinco seções principais: a Seção 1 traz a introdução e o escopo do projeto; a Seção 2 apresenta a descrição geral do produto e de seus usuários; a Seção 3 especifica os requisitos funcionais, não funcionais, as interfaces externas e os requisitos de dados; a Seção 4 apresenta o Diagrama de Casos de Uso; e a Seção 5 apresenta os Apêndices, com a visão da interface e a stack tecnológica utilizada.

2. DESCRIÇÃO GERAL

2.1 Perspectiva do Produto

O Mango Trading é uma aplicação web independente, desenvolvida com arquitetura desacoplada. O front-end e o back-end são aplicações independentes que se comunicam por meio de uma API REST, utilizando requisições HTTP e dados no formato JSON. No back-end, o sistema é organizado em camadas (Controller, Service e Repository), cada uma com responsabilidades específicas dentro do processamento das operações de compra, venda, gerenciamento de carteira e cálculo de patrimônio.

A organização de diretórios do back-end compreende, entre outros, os pacotes config, controller, dto, entity (models/entidades JPA), exception, repository, security e service. O front-end, por sua vez, é organizado em components, pages, services, hooks, contexts, routes e types, consumindo os dados fornecidos pela API REST do back-end.

2.2 Funções do Produto

- Cadastro e autenticação de usuários.

- Administração de empresas e ações fictícias (criação, edição, remoção e ativação/desativação).
- Compra e venda de ações utilizando saldo virtual.
- Acompanhamento da carteira de investimentos.
- Registro do histórico de transações.
- Cálculo do patrimônio total dos usuários.
- Sistema de ranking competitivo entre os jogadores.

2.3 Características dos Usuários

Visitante: pessoa não autenticada que acessa a aplicação antes de possuir uma conta. Possui acesso apenas às rotas públicas de cadastro e login, conforme detalhado na Seção 4.

Usuário Comum: usuário autenticado no sistema que pode visualizar as ações disponíveis, consultar preços, comprar e vender ações, visualizar sua carteira, consultar seu histórico de transações e seu patrimônio, além de consultar o ranking de jogadores.

Administrador: usuário autenticado responsável pelas funcionalidades de administração do sistema, podendo criar, editar e remover empresas e ações fictícias, ativar ou desativar ações, gerenciar eventos do mercado e visualizar os usuários cadastrados. As rotas administrativas são protegidas por meio do Spring Security.

2.4 Restrições Gerais

- O back-end deve ser desenvolvido em Java 17 ou superior, utilizando o framework Spring Boot.
- O front-end deve ser desenvolvido separadamente, utilizando React e TypeScript.
- O banco de dados utilizado deve ser o PostgreSQL, com persistência real dos dados.
- A arquitetura deve ser desacoplada, com comunicação entre front-end e back-end realizada por meio de uma API REST em formato JSON.

- O sistema não deve utilizar dinheiro real; todas as operações envolvem exclusivamente saldo virtual.
- As rotas administrativas devem ser protegidas utilizando o Spring Security.

2.5 Suposições e Dependências

Não há suposições ou dependências formalmente definidas no documento de planejamento do projeto até o presente momento. Esta seção deverá ser complementada assim que essas informações forem estabelecidas pela equipe.

3. REQUISITOS ESPECÍFICOS

3.1 Requisitos Funcionais

ID	Descrição	Prioridade
RF01	O sistema deve permitir o cadastro de usuários.	Alta
RF02	O sistema deve permitir que usuários realizem login.	Alta
RF03	O sistema deve possuir controle de acesso baseado em papéis.	Alta
RF04	O administrador deve poder realizar CRUD de ações.	Alta
RF05	O usuário deve poder visualizar as ações disponíveis.	Alta
RF06	O usuário deve poder comprar ações.	Alta
RF07	O usuário deve poder vender ações.	Alta

ID	Descrição	Prioridade
RF08	O sistema deve atualizar o saldo do usuário após operações.	Alta
RF09	O sistema deve manter uma carteira de investimentos para cada usuário.	Alta
RF10	O sistema deve registrar todas as transações.	Alta
RF11	O sistema deve calcular o patrimônio total do usuário.	Alta
RF12	O sistema deve apresentar um ranking de usuários.	Alta

Nota: a prioridade Alta foi atribuída a todos os requisitos funcionais por comporem o MVP (Produto Mínimo Viável) definido no planejamento do projeto.

3.2 Requisitos Não Funcionais

ID	Categoria	Descrição
RNF01	Tecnologia	O sistema deve utilizar Java 17 ou superior.
RNF02	Tecnologia	O back-end deve utilizar Spring Boot.
RNF03	Confiabilidade	O banco de dados deve possuir persistência real.
RNF04	Tecnologia	A aplicação deve utilizar um banco de dados relacional.

ID	Categoria	Descrição
RNF05	Interoperabilidade	A comunicação entre front-end e back-end deve ocorrer através de uma API REST.
RNF06	Segurança	O sistema deve possuir controle de autenticação e autorização.
RNF07	Acessibilidade	A interface deve atender requisitos essenciais de acessibilidade.
RNF08	Usabilidade	A interface deve ser responsiva.
RNF09	Manutenibilidade	O código deve possuir organização em camadas.

3.3 Interfaces Externas

Interface de Usuário: interface web construída com React, desenvolvida com HTML semântico (uso de tags como <header>, <nav>, <main>, <section>, <article> e <footer>), formulários com labels adequados e atributos ARIA (aria-label, aria-describedby), além de contraste de cores adequado, conforme as diretrizes WCAG AA consideradas no planejamento do projeto.

Interface de Software: comunicação entre o front-end (React) e o back-end (Spring Boot) realizada por meio de uma API REST, utilizando requisições HTTP e dados no formato JSON.

Interface de Hardware: não há interfaces de hardware específicas definidas no documento de planejamento do projeto.

3.4 Requisitos de Dados

As principais entidades do banco de dados e seus atributos primários compreendem:

User: id, name, email, password, balance, role, createdAt, updatedAt.

Stock: id, name, symbol, description, sector, currentPrice, volatility, active, createdAt, updatedAt.

Portfolio: id, userId, createdAt.

PortfolioItem: id, portfolioId, stockId, quantity, averagePrice.

Transaction: id, userId, stockId, type, quantity, price, createdAt.

StockPriceHistory: id, stockId, price, createdAt.

MarketEvent: id, title, description, impact, startDate, endDate, createdAt.

Quanto aos relacionamentos, User possui relação 1:1 com Portfolio e 1:N com Transaction; Portfolio possui relação 1:N com PortfolioItem; e Stock possui relação 1:N com PortfolioItem, Transaction e StockPriceHistory.

4. DIAGRAMA DE CASOS DE USO

Esta seção apresenta os atores do sistema Mango Trading e suas respectivas interações, com base nas permissões de acesso definidas no planejamento do projeto (arquitetura de segurança com Spring Security).

4.1 Atores e Permissões de Acesso

Visitante: pessoa não autenticada, com acesso apenas às rotas públicas de cadastro e login.

Usuário Comum: usuário autenticado, com acesso às funcionalidades de negociação, carteira e ranking.

Administrador: usuário autenticado com papel administrativo, com acesso às funcionalidades de gestão do mercado de ações.

4.2 Representação Gráfica

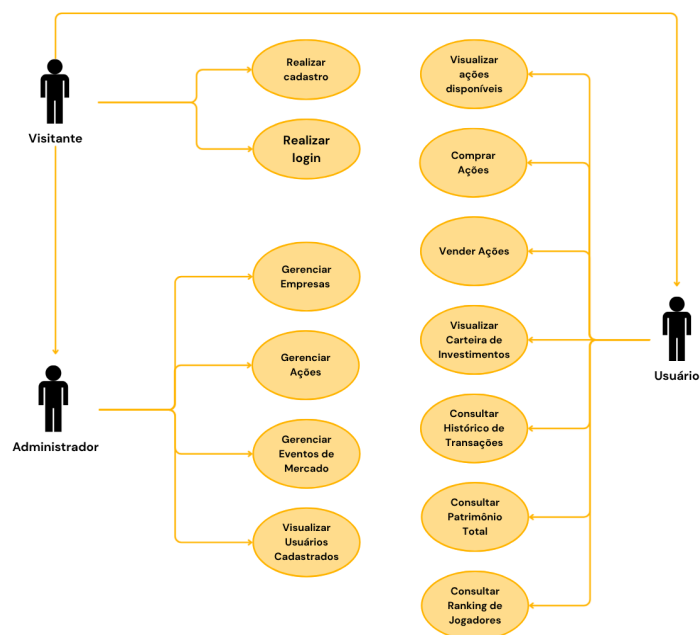


Figura 1 – Diagrama de Casos de Uso do sistema Mango Trading.

4.3 Casos de Uso por Ator

Visitante: Realizar cadastro; Realizar Login.

Usuário Comum: Visualizar Ações Disponíveis; Comprar Ações; Vender Ações; Visualizar Carteira de Investimentos; Consultar Histórico de Transações; Consultar Patrimônio Total; Consultar Ranking de Jogadores.

Administrador: Gerenciar Empresas; Gerenciar Ações; Gerenciar Eventos de Mercado; Visualizar Usuários Cadastrados.

5. APÊNDICES

A. Protótipos e Visão da Interface

Esta seção ainda não foi definida, bora 03.

B. Stack Tecnológica

Camada	Tecnologias Selecionadas
Back-end	Java 17 ou superior, Spring Boot, Spring Web, Spring Data JPA, Spring Security, Hibernate, Maven
Front-end	NextJs:Typescript, react, tailwindCss
Banco de Dados	PostgreSQL